Sprint Report 5

Team Plugged In
David Jarman, Marshall Gaucher, Kevin Worner
Project STBMF
February 13th – March 13th 2013

Initial Sprint Backlog

The main goal of this sprint was to have a mostly working product by the end of it, including a completed game server, a plugin with no AI, and showing movement updates on the virtual set top box. Team member vision was discussed and focused in one direction.

Server

1. Write callback methods for incoming messages
    - 1.1. Server Messages
        - 1.1.1. Update Game State
        - 1.1.2. Initialize Game Board
        - 1.1.3. Choose Items From List
    - 1.2. Set Top Box Messages
        - 1.2.1. Download Game List
        - 1.2.2. Request Resources
        - 1.2.3. Start New Game
        - 1.2.4. Send Move
        - 1.2.5. Forfeit Game
        - 1.2.6. Pause or Resume Game
        - 1.2.7. Get Active Game List
2. Fill in error messages for callback methods
3. Create error message class
4. Refactor to allow for dependency injection
    - 4.1. ICommunication
    - 4.2. IDatabase
    - 4.3. ISerializer
    - 4.4. IGameFactory
5. Write method to convert two full gamestates into an update gamestate
6. Load plugins
7. Break up big messages to send multiple packets
8. Create matchmaking system
9. Refactor IMPGameInterface to be called IMPBoardGameInterface
10. Research Moq and other object mocking frameworks for .NET

Virtual STB

1. Redo sendmove, forfeit, pause/resume logic
2. Udp resource handler for large files
3. Fix CRC bug
4. Refactor Game View
5. Refactor Item Selection Dialog
6. Add existing live communication and CRC lib
7. Fix request serializer bug

Checkers

1. Create checkers
2. Unit test checkers
    2.1. Moves
    2.2. Jumps
    2.3. Picked opponent piece
    2.4. Picked empty space
    2.5. Resource not null
    2.6. Init game board
    2.7. Init game state
    2.8. Picked red space
    2.9. Move on diagonal

Virtual APMAX

1. Integrate Logger into project

Common

1. Read C# coding standards
2. Prepare unit and regression testing discussion

Extra Work Completed

On the server, David began unit testing using Moq and dependency injection, where dependencies such as using the database, communicating over sockets, and serializing messages, are injected into the code using interfaces. This allows Moq to fake implementation and for much less work to be done to unit test portions of code.

The logic for checkers was done much quicker than expected, and was also fully unit tested by Kevin, which was not planned. This did not mean that the plugin was error free, but it did provide a very good basis for testing other parts of the project. Once the main logic was done, Kevin decided to begin implementing the A.I. for checkers, and had a working code base in a couple of days. He also made optimizations using Alpha Beta pruning, but there is still work to be done optimizing it further.

Not only did Marshall get updates to work on the virtual Set Top Box, he also updated the U.I. to more closely match what the actual Set Top Boxes look like.

Work Carried to Next Sprint

Server

There needs to be many more unit tests on the server, but it is now set up to more easily create tests. There are also still a couple of classes without documentation, and that will also be pushed onto the next sprint. At this point though, there will not be any more major additions to this component.

Checkers

There will be more testing and bug fixing in the next sprint for checkers. As bugs are fixed, there will also be more unit tests written. The AI portion also needs a few more optimizations. As of this writing, there are 55 unit tests backing checkers.

Virtual STB

There a couple of bugs that need to be taken care of yet, such as crashes when a second game is started after the first one finishes. Also taking the existing vSTB database class and changing the connection and query statements to match the existing database schema. This will provide one central database for all errors, warnings, and issues generated by the vSTB. The only major feature that still needs to be implemented is using a remote control and arrow keys to move the selected area around the board. Once all the features are added, Marshall will begin to write some unit tests.

Research/Code Experiments

Server Unit Testing

David researched Moq this sprint, which is a .NET mocking library for unit testing. Once Moq was finally integrated into the project (using NuGet), it became a challenge to write unit tests, since the database had to be connected and communications would fail when there was no client to receive the messages being sent. More research into writing unit testable code produced the concept of dependency injection. The idea is that rather than using the actual class name of the code to send messages or use the database, one would use an interface with all the same methods on it, and then the class you would normally use implements that interface. At runtime, there is no difference, but when unit testing, you can use libraries like Moq to “mock” the implementation of the dependency. For instance, when calling to the database to add a state, one would just mock the IDatabase interface to return true, and when you want to get the state back, you mock IDatabase again to return some phony object that you would normally expect at runtime. This allows for the unit tests to actually test the method, and not the dependencies that method uses.

Virtual STB Refactoring

While the server was being developed, Marshall was forced to stub methods and hardcode data until the server could produce real results that could be tested against. After many months of doing this, some refactoring had to be done to use real data from the server. David helped out with this by moving some of the protocol definitions into separate files and grouping those files into common folders. This improved some of the organization of the vSTB, as well as speeding up compilation time.

Virtual STB Drawing

To get checkers pieces to look like they are sitting on top of the board, Marshall had to look into drawing the images on top of each other, and doing so in an efficient way. The end result was a very nice looking board where the piece images could be dynamically drawn on top of the tile images. Additionally he looked into QT style sheets to simplify palette logic on vSTB GUI Buttons.

Virtual STB Unit Testing

Marshall has been conducting ongoing research into Qt's unit testing libraries. Qt's provided documentation is rather trivial and doesn't offer much explanation into the finer details of the provided libraries. Due to this, he has been looking into alternative tools that may simplify unit testing.

Checkers A.I.

Kevin has taken the Artificial Intelligence class on campus, so he was the obvious candidate to write the checkers AI portion. In only a couple of days he had a fully working analyzer that would look three moves ahead and decide which move to pick. This level of difficulty is fairly easy to beat, but any more moves ahead causes a drastic increase in computation. He then looked into more optimizations and remembered what he had learned about Alpha Beta pruning, and had that integrated in a short time. It did not help as much as he had hoped, but after some careful analysis, we decided that we would try some more optimizations next sprint.

Meetings

Each week, the team met with Scrum Master Brian Butterfield, and clarified our vision and gave updates as to our project. He would also help clarify on questions we had come up with, and occasionally offer tips as to how to implement certain portions.